

IA Aplicada com LLMs

Aula 05 — Confiabilidade

Onde paramos

Aula 01, nota 03: uma tabela com **oito modos de falha** de agentes.

Naquele momento era um mapa. Hoje cada linha vira código.

O índice deste deck é aquela tabela

Falha	Onde resolvemos
Laço	§6
Deriva de objetivo	§7
Alucinação de argumento	§3, §4
Ferramenta errada	deck anterior
Contexto estourado	próximo deck
Erro em cascata	§3, §8
Custo descontrolado	§1
Silêncio	§2, §10

O aviso que abre o deck

Nenhuma salvaguarda é grátis

- Orçamento apertado **mata tarefa legítima**
- Detector agressivo **interrompe quem progredia devagar**
- Confirmação em excesso **mata a autonomia**

Escolher a dose é engenharia, não receita.

Parte 1

Orçamento e terminação

max_passos=6 não é orçamento

Passo não é unidade de custo.

Passo	Tokens
consultar um id	~300
ler um documento de 30 páginas	~40.000

"No máximo 10 passos" = "no máximo entre 3 mil e 400 mil tokens".

Isso não é um limite.

Quatro moedas

```
@dataclass
class Orcamento:
    max_passos: int = 12
    max_tokens: int = 60_000
    max_reais: float = 0.50
    max_segundos: float = 120.0

    def excedido(self, estado) -> str | None:
        """Devolve QUAL teto estourou – o motivo importa."""
```

Quatro observações

É parâmetro, não constante. Triagem e investigação não têm o mesmo direito de gastar.

Reais vem da Aula 02 — é a moeda que o seu chefe entende.

Tempo de parede é o teto esquecido — pega a ferramenta travada, que não estoura nenhum dos outros três.

Devolve *qual* teto estourou — `bool` seria simples e inútil.

As quatro formas de terminar

1. RESPONDEU	devolveu sem tool_calls	✓ sucesso
2. ORCAMENTO	estourou um dos quatro tetos	✗ inconcluso
3. ERRO_FATAL	não há como continuar	✗ falha
4. HUMANO	pausou aguardando confirmação	suspenso

Na Aula 03: a 1ª era `return`, a 2ª era `raise`, a 3ª derrubava o processo e a 4ª não existia.

O trace grava sempre

```
finally:  
    registrar_trace(estado)      # inclusive nas falhas
```

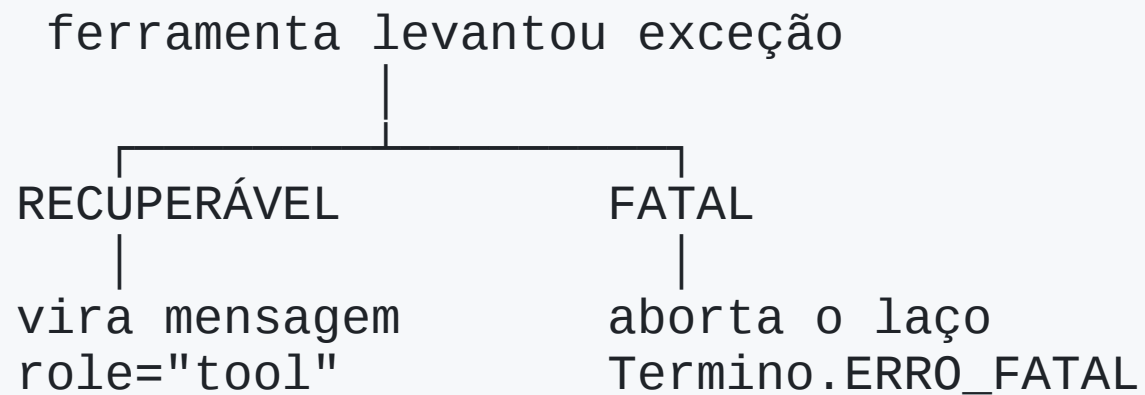
O `except` que registra só o sucesso é o `except` que garante que você nunca vai achar a causa.

A linha "**silêncio**" da tabela de falhas morre aqui.

Parte 2

Erro

A decisão mais importante do deck



Quem classifica é o seu código. Não o modelo.

Recuperável × fatal

Recuperável — o modelo contorna mudando a chamada:
argumento inválido, registro inexistente, formato errado

Fatal — nenhuma decisão do modelo resolve:
credencial, banco fora do ar, disco cheio

Devolver um fatal ao modelo = ele reformula educadamente
quatro vezes até o orçamento acabar.

A pergunta que classifica

Existe alguma chamada diferente que o modelo poderia fazer para contornar?

Se existe → **recuperável**, e o erro deve dizer qual é.

Na dúvida: origem no **argumento** = recuperável.

Origem na **infraestrutura** = fatal.

O corolário que ninguém enuncia

Aula 03: *"a descrição da ferramenta é prompt"*.

A mensagem de erro também é prompt

É o único texto que o modelo lê para decidir **como se corrigir**.

Três retornos de erro

```
{"erro": "falhou"} # inútil

{"erro": "ValueError: invalid literal for int()..."} # verdadeiro,
# e sem ação possível

{"erro": "formato de id inválido", # útil
 "esperado": "D seguido de 4 dígitos, ex: D-4471",
 "recebido": "4471",
 "sugestao": "chame listar_despesas"}
```

O terceiro responde: **o que errou · qual era o certo · o que fazer agora.**

Retry: onde vale e onde não

Falha	Quem resolve	Como
429 , 500 , timeout	seu código	backoff exponencial
argumento inválido	o modelo	erro volta como observação
credencial, serviço fora	ninguém	ErroFatal

Retry automático é para falha de **transporte**.

Falha de **conteúdo** volta para o modelo.

O anti-padrão

```
for tentativa in range(3):  
    try:  
        return FERRAMENTAS[nome](**argumentos)  
    except ErroRecuperavel:  
        continue          # os argumentos são os mesmos.  
                           # O erro será o mesmo.
```

Retry sem informação nova é só uma forma lenta de falhar.

Parte 3

Laço, deriva, escrita

max_passos limita o dano. Não detecta o laço.

Com teto de 12, um agente preso na 1ª volta gasta 12 passos e devolve o diagnóstico **errado**: "orçamento esgotado".

```
def assinatura(passo) -> tuple:  
    return (passo.ferramenta, json.dumps(passo.argumentos, sort_keys=True))
```

O `sort_keys=True` não é detalhe: sem ele, o detector não detecta nada.

Progresso nulo — a versão sutil

As chamadas **variam**. Nada avança.

```
consultar_politica → consultar_historico → consultar_politica  
→ consultar_despesa → consultar_politica → ...
```

Cada chamada é diferente da anterior. O **conjunto** se repete.

O que fazer ao detectar

1. **Injetar observação** — *"você já chamou isso e recebeu aquilo"*
(mais barata, e costuma bastar)
2. **Reduzir as ferramentas ativas** — força outro caminho
3. **Abortar** com motivo `laco_detectado` — diagnóstico honesto

Preço: detector agressivo dispara falso positivo em quem legitimamente varre uma lista. Comece frouxo.

Deriva de objetivo

Passo 30: o pedido original está cercado de 12.000 tokens.
É exatamente o **meio** do contexto (*lost in the middle*).

```
if estado.n_passos % REANCORAR_A_CADA == 0:  
    msgs.append({"role": "user",  
                "content": f"Lembrete do objetivo: {estado.objetivo}"})
```

Dezenas de tokens a cada 5 passos.

A melhor relação custo-benefício da aula.

Idempotência: agora não é mais hipótese

As salvaguardas que acabamos de adicionar **aumentam** a chance de repetir uma escrita:

- o **retry** reenvia o que já teve efeito
- o **checkpoint** retoma de um passo talvez já executado
- a **detecção de laço** deixa o agente continuar e repetir

A chave vem do conteúdo, não da tentativa

```
if (existente := PARECERES.get(chave)):  
    return {**existente, "ja_existia": True}
```

`uuid4()` a cada chamada **não** é chave de idempotência —
é um identificador novo por tentativa.

E o `ja_existia: True` é informação **para o modelo**:
ele descobre que já agiu.

Checkpoint

```
def salvar_checkpoint(estados):  
    caminho.write_text(json.dumps(asdict(estados), default=str))
```

Desativa: **confirmação assíncrona · retomada após queda · reprodução**

⚠ Salve **depois** de executar, nunca antes.

A idempotência fecha a fresta que sobra.

O exemplo que fecha o deck

SEM detector, erro inútil:

```
passos 0..11  consultar_historico {"funcionario": "F-88"} -> erro  
TERMINO: orcamento_esgotado | 14.200 tokens | R$ 0,036
```

COM erro que ensina:

```
passo 0  -> {"erro": "...", "esperado": "F-088",  
            "sugestao": "chame listar_funcionarios"}  
passo 1  consultar_historico {"funcionario": "F-088"} -> ok  
TERMINO: respondeu | 1.480 tokens | R$ 0,004
```

O detector **nem disparou**. Quem resolveu foi o texto do erro.

O subproduto que vale mais que o produto

Para detectar laço → guardamos ferramenta e argumentos.

Para o orçamento → tokens e custo.

Para o término → o motivo.

Isso é um trace

Matéria-prima das aulas de **observabilidade** e **evals**.

Não se avalia o que não se rastreia.

Síntese

- `max_passos` não é orçamento: meça em **4 moedas**, e passe como parâmetro
- Quatro formas de terminar, todas registradas — trace no `finally`
- Erro é **recuperável** ou **fatal**; quem classifica é o seu código
- **A mensagem de erro é prompt**
- Retry para **transporte**; conteúdo volta para o modelo
- Detecte laço e **progresso nulo**; intervenha antes de abortar
- Reancore o objetivo a cada N passos
- Chave de idempotência do **conteúdo**; checkpoint **depois** de executar
- Nada disso é grátis